

# Assignment 2 - uaccess

April 1, 2025

## 1 Background

If the kernel wants to access the user address space, we generally do not allow it to directly perform memory operations on user addresses, such as pointer dereferencing and memcpy. Instead, it needs to use specialized functions designed for accessing user space, such as `copy_from_user` and `copy_to_user`. We refer to these functions as uaccess primitives (user space access primitives).

During system calls, user programs pass user addresses as needed and instruct the kernel to read or write to them. For example, in the `read(2)` and `write(2)` system calls, the user provides a buffer `buf` and its length `count`, indicating that the kernel is allowed to read or write up to `count` bytes of data.

```
1 ssize_t read(int fd, void buf[.count], size_t count);
2 ssize_t write(int fd, const void buf[.count], size_t count);
```

In *console.c*, the handling of the `sys_write` system call is as follows (simplified version): First, we allocate a buffer of length `len` in kernel space and use `copy_from_user` to copy the data to be printed from user space. Finally, we print the data to the serial port.

```
1 int64 user_console_write(uint64 __user buf, int64 len) {
2     struct proc *p = curr_proc();
3     struct mm *mm = p->mm;
4
5     char *kbuf = kmalloc(len);
6     copy_from_user(mm, kbuf, buf, len);
7
8     for (int64 i = 0; i < len; i++) {
9         consputc(kbuf[i]);
10    }
11
12    return len;
13 }
```

Why do we need uaccess primitives to operate on user memory instead of directly dereferencing user pointers?

- User-provided pointers are untrusted. Using primitives ensures that each access checks whether the pointer is valid.

---

Build commit hash:

- Directly dereferencing user pointers without checks is highly dangerous. Users may intentionally pass a valid kernel address instead of a user address. In such cases, direct dereferencing would lead to access to kernel memory. Using primitives enforces checks to avoid such vulnerabilities.
- If kernel-mode code could directly access user memory, the compiler might optimize kernel code for memory access, leading to security issues like Double Fetch. If interested, we recommend watching: [Windows Kernel Deep Dive](#) [How Can Compiler Optimizations Introduce Potential Risks?](#)

## 2 Problem Description

In the current xv6 kernel, the kernel page table and user page tables are isolated from each other. The kernel page table (`kernel_pagetable`) does not include the page table entries (`p->mm->pgt`) of any user process, and vice versa. In `copy_from_user`, we first manually translate the user virtual address to a physical address (via the `walkaddr` method). Since all user-space memory is allocated by the `kallocpage` method in the `kalloc` module, these physical addresses reside within the Kernel Direct Mapping region of physical memory. Therefore, we can convert them into kernel virtual addresses (KVA) for access:

```
1 // Copy from user to kernel.
2 // Copy len bytes to dst from virtual address srcva in a given page table.
3 // Return 0 on success, -1 on error.
4 int copy_from_user(struct mm* mm, char *dst, uint64 __user srcva, uint64 len)
5 {
6     uint64 n, va0, pa0;
7
8     while (len > 0) {
9         va0 = PGROUNDDOWN(srcva);
10        pa0 = walkaddr(mm, va0);
11        if (pa0 == 0)
12            return -1;
13        n = PGSIZE - (srcva - va0);
14        if (n > len)
15            n = len;
16        memmove(dst, (void *) (PA_TO_KVA(pa0) + (srcva - va0)), n);
17
18        len -= n;
19        dst += n;
20        srcva = va0 + PGSIZE;
21    }
22    return 0;
23 }
```

This approach is clearly inefficient. Address translation should be performed by the CPU's MMU, and the results should be cached in the TLB. However, the current implementation of `copy_from_user` requires at least one page table walk for each call.

In this assignment, you are tasked with modifying the given code to improve the `uaccess` primitives, enabling them to use the CPU's page table mechanism for address translation while ensuring the safety of the `uaccess` primitives. This includes checking user-provided addresses and preventing erroneous memory accesses.

**This assignment is worth 4 points, with 4 Checkpoints and a report.** Each Checkpoint is worth 1 point, and the report is mandatory but not graded. Your report should meet the following requirements:

- A PDF-formatted report.
- Screenshots of successful runs for each checkpoint.
- You do not need to answer the reflection questions in the report.
- Record the total time spent on this assignment.
- (Optional) Describe any difficulties encountered or additional guidance you think this assignment needs.
- (Optional) Share your approach to completing this assignment.
- Keep it concise.

**Requirements** You should use `make run` to start a single-core xv6. For this assignment, we provide `usertest`. After booting the kernel, you should see an entry `test_uaccess` under the `applist:` prompt. You need to pass `test_uaccess 1-4`. Stable passing is considered as passing the Checkpoint.

Note: Checkpoints 2-4 are only scored if Checkpoint 1 has already been passed.

## 3 Guidance Steps

### 3.1 Checkpoint. 1

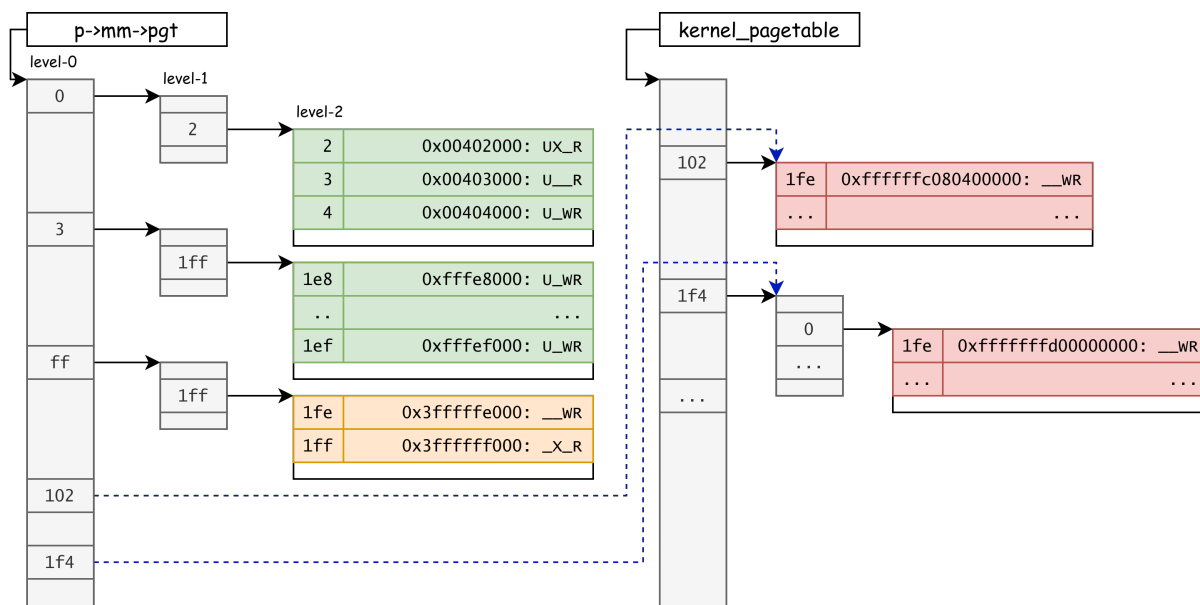
Modify xv6's user and kernel page table model so that:

- The user page table includes the entire kernel page table (`kernel_pagetable`). (The PTEs in the kernel page table do not have the `PTE_U` permission bit.)
- Kernel threads use the user page table (which should include the kernel page table) instead of the kernel page table.

Additionally, modify Context Switch and Trampoline:

- When performing a Context Switch with `swtch`, switch to the respective kernel thread's page table (`p->mm->pgt`).
- When switching between user mode and kernel mode via trampoline, retain the user page table and do not switch to the kernel page table `kernel_satp`.
- For the scheduler without user space, directly use the `kernel_pagetable` kernel page table.

**Hint.** The kernel page table remains unchanged after kernel initialization. You can directly copy the PTEs from the first-level page table of the kernel page table to the user page table, pointing them to the second-level page table of the kernel page table.



**Hint.** After modifying `satp`, use `sfence.vma zero, zero` to flush all TLB entries.

**Hint.** The scheduler is never the first target of `swtch`.

**Checkpoint Passed.** You can use `make run` and `make runsmp` to boot the kernel normally. In the `sh` shell, execute `test_uaccess 1`, and you should see the following output:

```
1 sh >> test_uaccess 1
2 -> checkpoint 1 passed
```

## 3.2 Checkpoint. 2

Replace all `while(...)` loops in the `copy_to_user` and `copy_from_user` functions in `uaccess.c` with direct `memmove` calls. (You do not need to modify the `copystr_from_user` function; our test cases do not cover it.) For example:

```
int copy_to_user(struct mm* mm, uint64 __user dstva, char *src, uint64 len) {
    memmove((void *)dstva, src, len);
    return 0;
}

int copy_from_user(struct mm* mm, char *dst, uint64 __user srcva, uint64 len)
{
    memmove(dst, (void *)srcva, len);
    return 0;
}
```

Boot the kernel with `make run`. You should encounter a kernel exception with the cause 15 - Store/AMO page fault. The backtrace of this exception is as follows (refer to the Debug section for how to backtrace the Trapframe):

```
1 (qemu) gef bt
2 #0 0xffffffff802065a0 in memmove (dst=dst@entry=0xffffffffe00001ee0, src=0x404028, n=n@entry=0x12) at
   os/string.c:36
3 #1 0xffffffff80207f70 in copy_from_user (mm=mm@entry=0xffffffffd000fffc8, dst=dst@entry=0
   xffffffffe00001ee0 "\310\377\017", srcva=<optimized out>,
4     len=len@entry=0x12) at os/uaccess.c:39
5 #2 0xffffffff8020082c in user_console_write (buf=<optimized out>, len=0x12) at os/console.c:178
6 #3 0xffffffff80207134 in sys_write (fd=<optimized out>, va=<optimized out>, len=<optimized out>) at
   os/syscall.c:142
7 #4 syscall () at os/syscall.c:186
8 #5 0xffffffff80207d44 in usertrap () at os/trap.c:142
9 #6 0x00000000004021d8 in ?? ()
```

Read the privileged manual *4.1.1.2 Memory Privilege in sstatus Register* about the SUM flag to understand why this Page Fault occurs and resolve the issue based on the documentation.

**Checkpoint Passed.** Pass `test_uaccess 2`.

## 3.3 Checkpoint. 3

Execute `test_uaccess 3` in the `sh` shell. Its source code is located in `user/src/test_uaccess.c`, and the kernel code for `ktest` is in `os/ktest/ktest_syscall.c`. `test_uaccess 3` intentionally passes a kernel address instead of a user address when calling `write`.

Modify the code in `uaccess.c` to use `access_ok` to check whether addresses labeled with `__user` are indeed within the user space address region. Note that the `Trapframe` and `Trampoline` are not addresses accessible to users. **Do not** use the page table (`p->mm->pgt`) to determine address validity. You do not need to check whether the address is a mapped virtual address.

**Hint.** In our OS design, user addresses are in the lower 256 GiB of the total 512 GiB address space (`0x0000_0000_0000_0000` to `0x0000_003f_ffff_ffff`), while kernel addresses are in the higher region. You can compare the address value to determine if it is a valid user-space address.

**Checkpoint Passed.** Pass `test_uaccess 3`.

### 3.4 Checkpoint. 4

Execute `test_uaccess 4` in the `sh` shell. You will encounter a Kernel Panic, and the `stval` value is an unmapped virtual address, `0x8000_0000`.

Modify the kernel code so that if a Page Fault occurs during `uaccess` **at a user address**, the process exits with `exit(-9)` instead of causing a Kernel Panic.

**Hint.** When transitioning from user mode to kernel mode via a `syscall`, `stvec` is set to `kernel_trap`. In the original design, `kernel_trap` should never handle kernel exceptions but directly trigger a Kernel Panic.

```
if (cause & SCAUSE_INTERRUPT) {
    // handle interrupt
} else {
    // kernel exception, unexpected.
    goto kernel_panic;
}
// ...
kernel_panic:
    errorf("==== Kernel Panic =====");
    print_sysregs(true);
    print_ktrapframe(ktf);
    panic("kernel panic");
```

**Hint.** When calling uaccess functions like `copy_from_user`, we hold `p->mm->lock` and release it after the function returns.

However, a Page Fault exception interrupts the normal function call flow. The flow is as follows:

```
// enter syscall (write), then user_console_write
acquire(&p->mm->lock);
copy_from_user(p->mm, dst, src, len);
{
    // copy_from_user function body:
    memmove(mm, dst, src, len);
    {
        // memmove function body:
        while (n-- > 0)
            *d++ = *s++;    // <-- Page Fault Here.
    }
}
release(&p->mm->lock);
// return syscall
```

**Checkpoint Passed.** Pass `test_uaccess` 4. You may not pass `proctest`, but this is not required for this assignment.

## 4 Submission

Place your report in the same directory as this PDF file and run `make handin`. This will generate a `handin.zip` file. Upload this file to Blackboard.

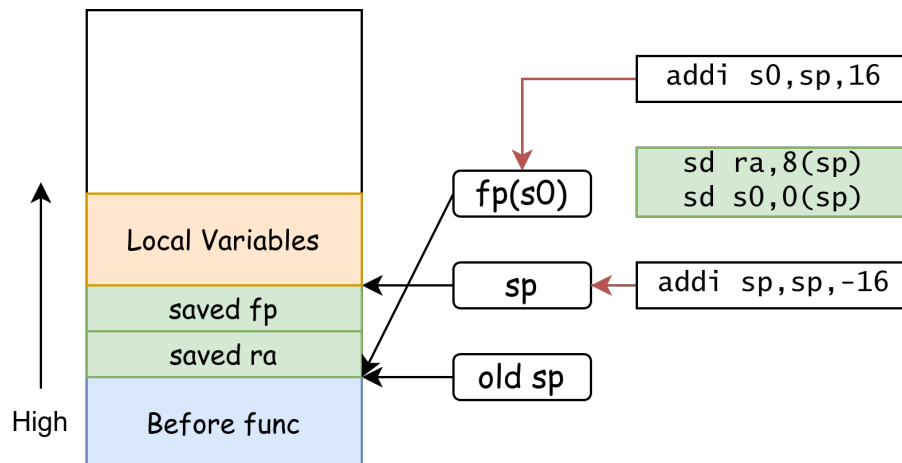
## 5 Debug

### 5.1 Stack Layout

We have learned about the structure of the stack in many computer courses. When a function is called, it constructs a stack frame on the stack. The function prologue pushes the return address `ra` and the current frame pointer value onto the stack and assigns the current stack pointer to the frame pointer register. The function epilogue restores `ra` and the `fp` from before the function was called.

On the RISC-V platform, the stack pointer is `sp`, and the Frame Pointer is `fp` (also `s0`).

```
some_func:
    addi    sp,sp,-16
    sd      ra,8(sp)
    sd      s0,0(sp)
    addi    s0,sp,16
    # function body
    ld      ra,8(sp)
    ld      s0,0(sp)
    addi    sp,sp,16
    jr      ra
```



### 5.2 Backtrace

We notice that the region pointed to by `fp` contains the previous `fp` value, so we can find all past stack frames using the current `fp`! This is the principle behind the `backtrace (bt)` command in `gdb`.

However, the `ktrapframe` constructed in the Trap Handler does not fully follow this design. If we want to backtrace the cause of a Trap in the context of a Kernel Trap, we need to make a small adjustment.

Run `make debug` and `gdb-multiarch`, and enter `gef`. Set a breakpoint at the `kernel_panic` label in `kernel_trap`: `b kernel_trap:kernel_panic`. Use `continue` to proceed.

When the breakpoint is hit, use the bt command to backtrace:

```
1 (qemu) gef bt
2 #0 kernel_trap (ktf=0xffffffffe00001de0) at os/trap.c:90
3 #1 0xffffffff802000b4 in kernel_trap_entry () at os/entry.S:55
4 Backtrace stopped: frame did not save the PC
```

Change the fp register to the fp saved in the ktf structure: set \$s0 = ktf->s0.

Then, we can backtrace in the context of the interrupt frame.

```
1 (qemu) gef bt
2 #0 kernel_trap (ktf=0xffffffffe00001de0) at os/trap.c:90
3 #1 0xffffffff802084e8 in copy_from_user (mm=mm@entry=0xffffffffd000fffc8, dst=dst@entry=0x0,
4   srcva=srcva@entry=0x404028, len=len@entry=0x0) at os/uaccess.c:36
5 #2 0xffffffff8020084c in user_console_write (buf=0x404028, len=0x0) at os/console.c:187
6 #3 0xffffffff802076e8 in sys_write (fd=<optimized out>, va=<optimized out>, len=<optimized out>) at os/
   syscall.c:180
7 #4 syscall () at os/syscall.c:227
8 #5 0xffffffff8020831c in usertrap () at os/trap.c:183
9 #6 0x0000000000402184 in ?? ()
```

Additionally, you can use bt -full to print local variables saved on the stack.

```
1 (qemu) gef bt -full
2 #0 kernel_trap (ktf=0xffffffffe00001de0) at os/trap.c:90
3   cause = 0xd
4   exception_code = 0xd
5   __func__ = "kernel_trap"
6 #1 0xffffffff802084e8 in copy_from_user (mm=mm@entry=0xffffffffd000fffc8, dst=dst@entry=0x0,
7   srcva=srcva@entry=0x404028, len=len@entry=0x0) at os/uaccess.c:36
8 No locals.
9 #2 0xffffffff8020084c in user_console_write (buf=0x404028, len=0x0) at os/console.c:187
10   ret = <optimized out>
11   p = 0xffffffffd0201ef10
12   mm = 0xffffffffd000fffc8
13   kbuf = 0x0
14 #3 0xffffffff802076e8 in sys_write (fd=<optimized out>, va=<optimized out>, len=<optimized out>) at os/
   syscall.c:180
15 No locals.
16 #4 syscall () at os/syscall.c:227
17   trapframe = 0xffffffffc080d05000
18   id = 0x17
19   ret = <optimized out>
20   args = {0x1, 0x404028, 0x12, 0xd, 0xa, 0xa}
21   __func__ = "syscall"
22 #5 0xffffffff8020831c in usertrap () at os/trap.c:183
23   which_dev = 0x0
24   p = 0xffffffffd0201ef10
25   trapframe = 0xffffffffc080d05000
26   cause = <optimized out>
27 #6 0x0000000000402184 in ?? ()
```